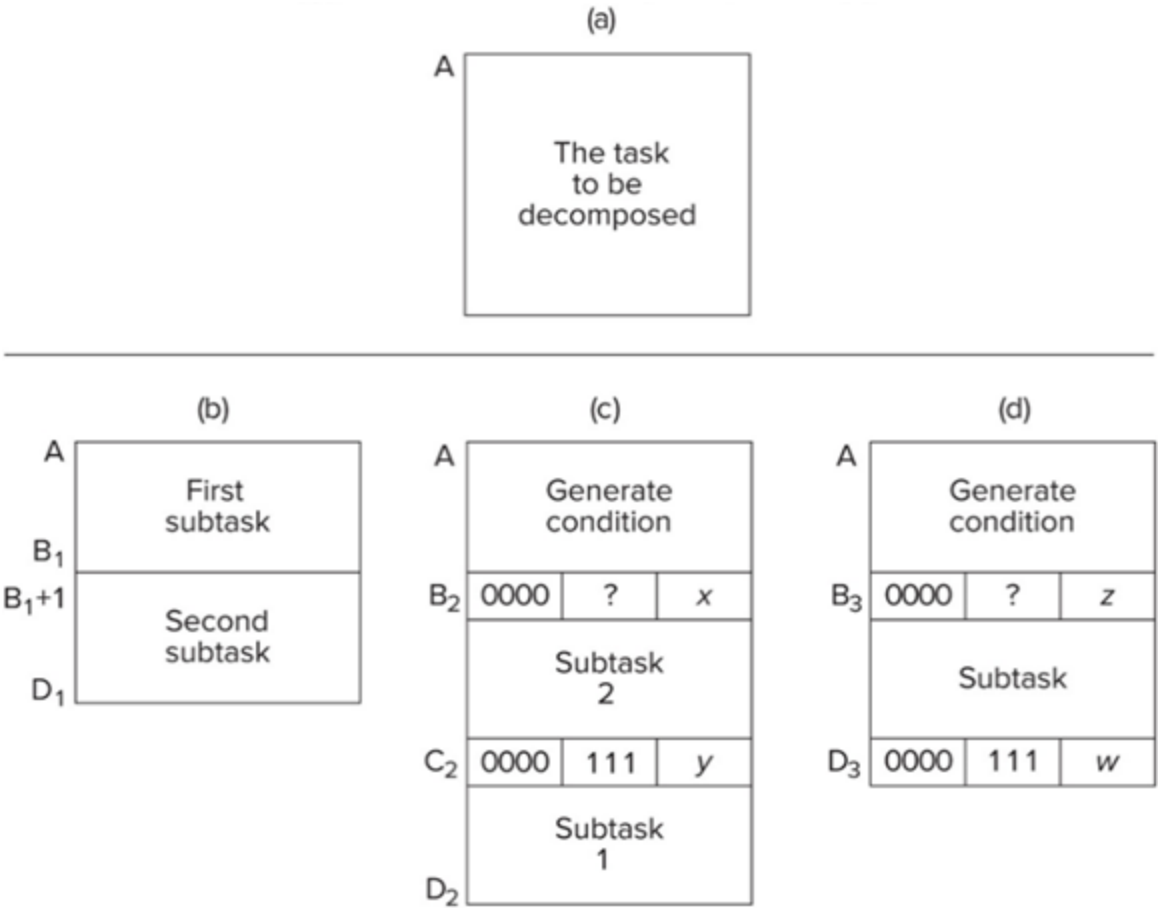
Problem Solving

Structured Programming:
The transformation from a problem statement to creating a solution via a software program involves two steps of transformation. The first transformation happens when we create an algorithm to solve the problem. Next, this algorithm is translated into a program using a computer language. This is step 2 of the transformation journey. It is in the “Problem to program” journey that the structured programming approach finds its importance.
Structured programming methodology is also called systematic decomposition, as the methodology revolves around systematically breaking a larger task or problem into smaller ones. We follow a stepwise refinement process until we have decomposed the original problem into multiple simple and smaller problems. We then identify clear solutions for the smaller problems and then stitch these solutions altogether to solve the original problem.
Sequential, Conditional and Iterative (SCI) Constructs:
Sequential, conditional, and iterative constructs are used in structured programming using the systematic decomposition methodology. The stepwise refinement process of the systematic decomposition methodology involves taking a larger unit of work and replacing it with a construct that correctly decomposes it. Three types of constructs are used for this purpose: sequential, conditional, and iterative.
The sequential construct is used when a larger task can be taken and subdivided into two subtasks that execute sequentially, that is, one subtask after the other. The first subtask is completely executed before the computer executes the subsequent subtask. The conditional construct is used when a larger task requires performing one of the two possible subtasks but not both. The subtask to be executed depends upon an evaluated condition to decide which of the two subtasks should be executed. If the condition evaluates to True, subtask 1 gets executed, but if the condition evaluates to False, subtask 2 gets executed. Note that any of the two subtasks could even be empty, meaning that nothing needs to be done at all as part of that subtask. Last, the iterative construct is used when a larger task involves executing a subtask multiple times until a condition is *True*. In other words, until a specified condition holds *True*, we keep executing a subtask iteratively until the condition evaluates to *False*. In this case, the computer moves ahead to the next step.
Implementing SCI Using LC-3:
The sequential construct is the simplest to implement, as it requires no special handling or control instruction to be implemented. The larger task is decomposed into two subtasks, each with its own instructions. As shown in the diagram below, subtask 1 instructions span from address capital A to address Capital B1 in the diagram. These instructions are executed first. After execution of subtask 1, the program counter normally increments to the instruction at capital B1+1 and starts to execute subtask 2 until address capital D1.



Source: Patt, Yale N., and Sanjay J. Patel, *Introduction to Computing Systems (From Bits and Gates to C/C++ and Beyond),* McGraw Hill, 3rd Edition

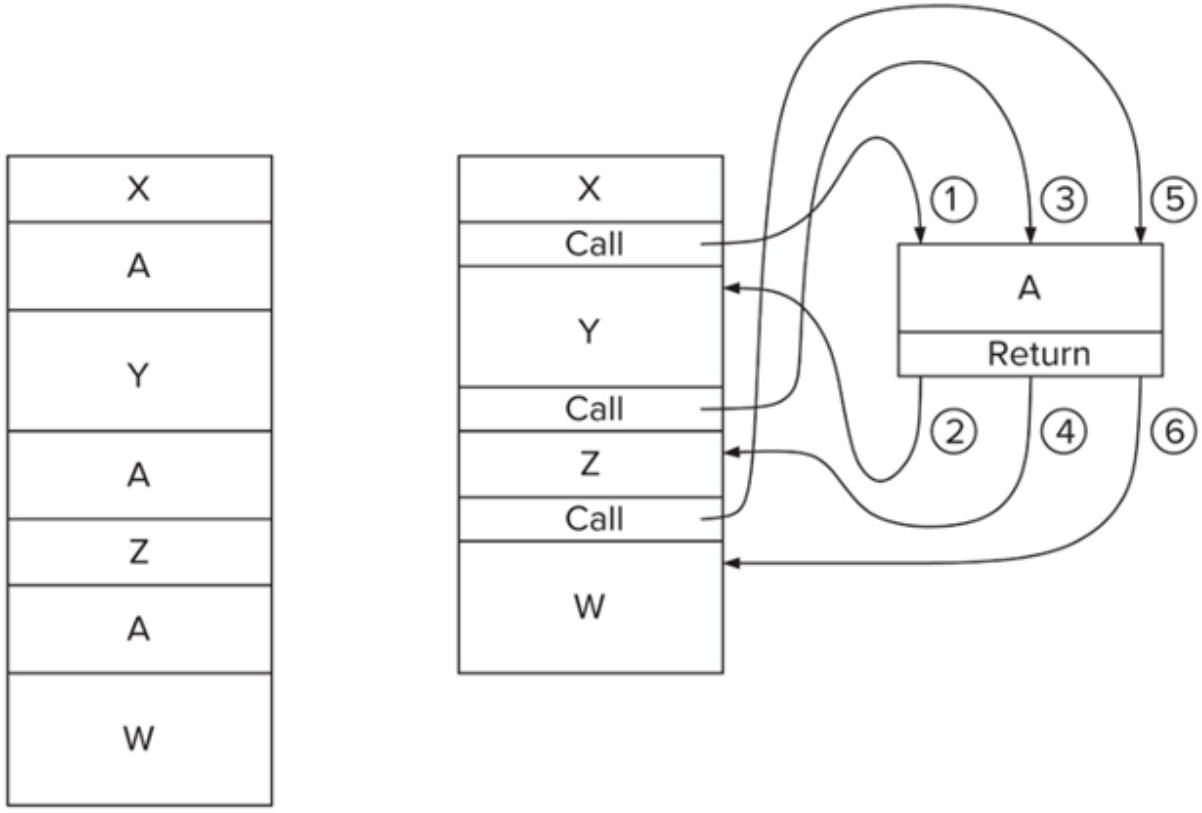
Next, let us understand how the conditional construct is implemented in the LC-3. Here, we first generate a condition that results in the setting of the condition codes. The condition is next tested by a conditional branch instruction at address capital B2. Let us assume that the condition is evaluated to be TRUE. In this case, the program counter will get set to Address capital C2+1, making use of the PC offset 'x' set appropriately in the instruction, and thus subtask 1 will get executed. On the other hand, if the condition had been evaluated to FALSE, the program counter would simply have continued to execute the next instruction at Address capital B2+1, and subtask 2 would get executed. When subtask 2 is completely executed, it terminates with a branch instruction at address capital C2. This is an unconditional branch instruction since the n, p, and z flag bits are set to all ones in the instruction. This results in the program counter unconditionally jumping to address capital D2+1 using the PC offset 'y' set appropriately in the branch instruction. This is how the conditional construct can be implemented in LC-3 using the LC-3 branch instruction.

Lastly, let us look at how the Iterative construct can be implemented in the LC-3. Just like the Conditional construct, the first thing is to generate the condition that will set the condition codes. At the end of this, a conditional branch statement checks for the required condition. Now, in the case of the iterative construct, assume that we wish to execute the subtask as long as the condition is evaluated to be TRUE. So, in the branch instruction, we instead check for the condition to be FALSE and set the program counter to address capital D3+1 when the condition does evaluate to FALSE. However, as long as the condition evaluates to TRUE, the program counter naturally increments and

executes the instructions of the subtask. At the end of the subtask, we have an unconditional branch statement that takes us back to address A to again generate and test the condition. The process keeps repeating till the condition eventually evaluates to FALSE, in which case, the program counter will finally move to address capital D3+1.This is how the Iterative construct can be implemented in LC-3 using the LC-3 branch instruction.

Subroutines:

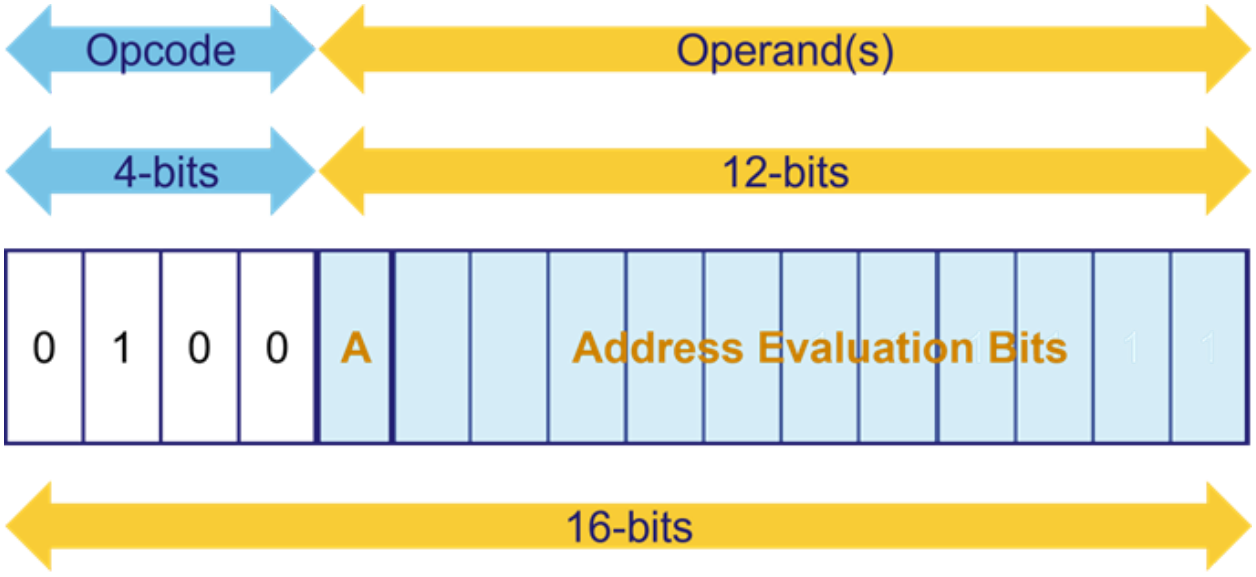
Sometimes, a software program needs to include the same set of instructions in many places in the code. Rather than repeating the same code multiple times at different locations in the software program, subroutines are used to write the code once and invoke it from different places. This introduces many benefits, including code reusability, simplicity, and modularity.



(a) Without subroutines (b) With subroutines

Source: Patt, Yale N., and Sanjay J. Patel, *Introduction to Computing Systems (From Bits and Gates to C/C++ and Beyond),* McGraw Hill, 3rd Edition

LC-3 provides the JSR and the JSRR control instruction to call or jump to a subroutine. It also includes the RET instruction, which is used to return back from the sub-routine to the main program. The format for the JSR and the JSRR control instruction is depicted in the diagram. Depending on which addressing mode is used, the LC-3 assembler provides two different mnemonic names for the same opcode, JSR and JSRR. JSR uses the PC-Offset addressing mode, while the JSRR uses the Base+Offset addressing mode. Using a base+offset addressing mode allows the subroutine to be anywhere in memory.



Name	A	Addressing Mode
JSR	1	PC-Offset
JSRR	0	Base+Offset

Before the software program can jump to a sub-routine, the value of the incremented program counter is stored in register R7. And when the sub-routine is executed, the program can jump back to the main program sequence using the address stored in register R7. This saving and restoring of the program counter is implicitly handled by the JSR and JSRR instructions.

LC-3 Assembly Language:

In general, any instruction code line in an assembly language code is of the following format:

LABEL OPCODE OPERANDS;Comments

LABEL, followed by the OPCODE symbolic name, followed by the OPERANDS name, and followed by any COMMENTS that can be written after using a semi-colon. While the OPCODE and OPERANDS are mandatory in the instruction, the LABEL and COMMENTS in any instruction are optional. While comments are general English language statements explaining to a reader what the instruction is supposed to do, the label put at the beginning of the code line represents a symbolic name given to the memory address corresponding to that line of code. In the case of a branch or jump instruction, we can simply use this label name to specify the memory address of the instruction we wish to branch or jump to.

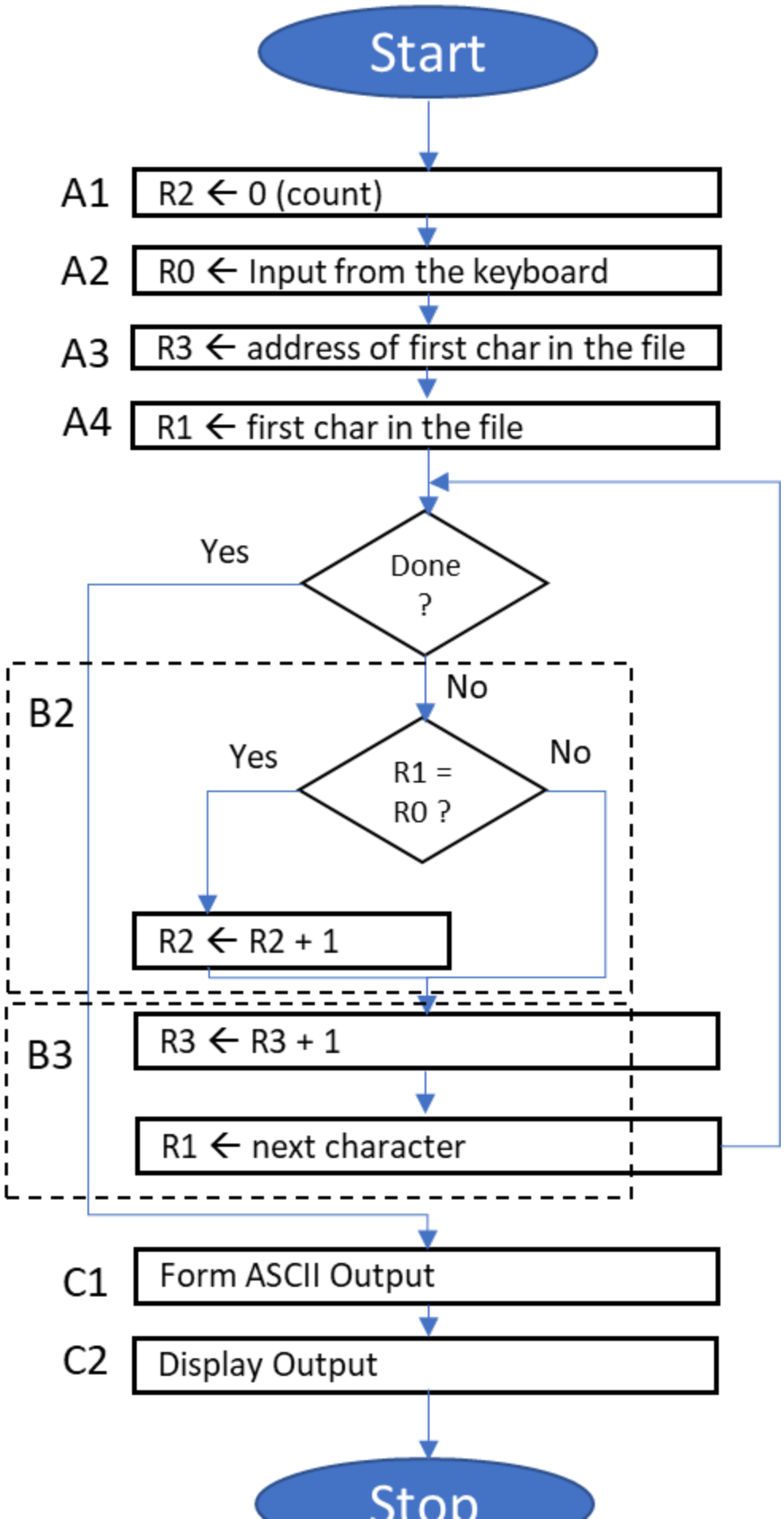
The opcodes are given a symbolic name for every instruction in the ISA. Examples: ADD, NOT, AND, LD, LDR, BRz, and others. Notice the last name, the BR stands for branch, and the nzp flag bits or condition codes become part of the opcode.

Operands, as we know, can be of many types. Register operands are specified using a capital R followed by the register number – 1, 2, 3, and so on. Numbers can be represented using either decimal notation or hexadecimal notation by prefixing them with # or x, respectively. A memory address location can be specified using the label.

Any line of code or text in an assembly language program is either an instruction of the format we just discussed or a line that could only include a comment, or the third possibility is to encounter an assembler directive, also sometimes called a Pseudo-op.
Assembler directives are also known as Pseudo-ops, indicative of the fact that they are not actual instructions to be executed. Instead, they provide some directives to the assembler. Recollect that an assembler is a utility that takes the assembly code and translates it into machine-level code that can be executed on the computer. Directives help provide some inputs from the programmer to the assembler tool in relation to the software program.
The table here summarizes the assembler directives and their uses. Just like an instruction, assembler directives also have opcodes and operands. But the opcode starts with a dot, differentiating it from an actual instruction of the ISA.

Opcode	Operand	Meaning
.ORIG	address	starting address of program
.END		end of program
.BLKW	n	allocate n words of memory
.FILL	n	initialize next location in program with value n
.STRINGZ	n-character string	allocate n+1 memory locations, initialize with the n string characters and null terminator

Example of Problem-Solving Using LC-3 Programming:
Our problem is to count the number of occurrences of a character in a pre-defined or given file. The user inputs the character via the keyboard, and we then want to display that count on the output monitor.
Systematic decomposition using structured programming techniques gives us the following flowchart to solve the problem.


Start
A1
R2 ← 0 (count)
A2
R0 ← Input from the keyboard
A3
R3 ← address of first char in the file
A4
R1 ← first char in the file
Yes
Done ?
No
B2
Yes
R1 = R0 ?
No
R2 ← R2 + 1
B3
R3 ← R3 + 1
R1 ← next character
C1
Form ASCII Output
C2
Display Output
Stop

Below is the LC-3 machine-level code for implementing the above flow chart:

Address	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
x3000	0	1	0	1	0	1	0	0	1	0	1	0	0	0	0	0	`R2 <- 0`
x3001	0	0	1	0	0	1	1	0	0	0	0	1	0	0	0	0	`R3 <- M[x3012]`
x3002	1	1	1	1	0	0	0	0	0	0	1	0	0	0	1	1	`TRAP x23`
x3003	0	1	1	0	0	0	1	0	1	1	0	0	0	0	0	0	`R1 <- M[R3]`
x3004	0	0	0	1	1	0	0	0	0	1	1	1	1	1	0	0	`R4 <- R1-4`
x3005	0	0	0	0	0	1	0	0	0	0	0	0	1	0	0	0	`BRz x300E`
x3006	1	0	0	1	0	0	1	0	0	1	1	1	1	1	1	1	`R1 <- NOT R1`
x3007	0	0	0	1	0	0	1	0	0	1	1	0	0	0	0	1	`R1 <- R1 + 1`
x3008	0	0	0	1	0	0	1	0	0	1	0	0	0	0	0	0	`R1 <- R1 + R0`
x3009	0	0	0	0	1	0	1	0	0	0	0	0	0	0	0	1	`BRnp x300B`
x300A	0	0	0	1	0	1	0	0	1	0	1	0	0	0	0	1	`R2 <- R2 + 1`
x300B	0	0	0	1	0	1	1	0	1	1	1	0	0	0	0	1	`R3 <- R3 + 1`
x300C	0	1	1	0	0	0	1	0	1	1	0	0	0	0	0	0	`R1 <- M[R3]`
x300D	0	0	0	0	1	1	1	1	1	1	1	1	0	1	1	0	`BRnzp x3004`
x300E	0	0	1	0	0	0	0	0	0	0	0	0	0	1	0	0	`R0 <- M[x3013]`
x300F	0	0	0	1	0	0	0	0	0	0	0	0	0	0	1	0	`R0 <- R0 + R2`
x3010	1	1	1	1	0	0	0	0	0	0	1	0	0	0	0	1	`TRAP x21`
x3011	1	1	1	1	0	0	0	0	0	0	1	0	0	1	0	1	`TRAP x25`
x3012	Starting address of file																
x3013	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0	`ASCII TEMPLATE`

Source: Patt, Yale N., and Sanjay J. Patel, *Introduction to Computing Systems (From Bits and Gates to C/C++ and Beyond),* McGraw Hill, 3rd Edition

The same code can be written using assembly language, as shown below:

```
; Initialization
                .ORIG       x3000
                AND         R2, R2, #0      ; R2 is counter, initialize to 0
                LD          R3, PTR         ; R3 is pointer to characters
                TRAP        x23             ; R0 gets character input
                LDR         R1, R3, #0      ; R1 gets the next character
;
; Test character for end of file
;
TEST            ADD         R4, R1, #-4     ; Test for EOT
                BRz         OUTPUT          ; If done, prepare the output
;
; Test character for match. If a match, increment count.
                NOT         R1, R1
                ADD         R1, R1, #1      ; R1 ← -R1
                ADD         R1, R1, R0      ; R1 ← R0 – R1. If R1=0, a match!
                BRnp        GETCHAR         ; If no match, do not increment
                ADD         R2, R2, #1
;
; Get next character from the file
;
GETCHAR         ADD         R3, R3, #1      ; Increment the pointer
                LDR         R1, R3, #0      ; R1 gets the next character to test
                BRnzp       TEST
;
; Output the count.
;
OUTPUT          LD          R0, ASCII       ; Load the ASCII template
                ADD         R0, R0, R2      ; Convert binary to ASCII
                TRAP        x21             ; ASCII code in R0 is displayed
                TRAP        x25             ; Halt machine
;
; Storage for pointer and ASCII template
;
ASCII           .FILL       x0030
PTR             .FILL       x4000
                .END
```